

Module 6 Week B — Integration Task: Semantic Pipeline

This is the Module 6 culminating deliverable. You will bring together both weeks of NLP Foundations – Week A's text preprocessing and named entity recognition with Week B's embedding representations – into a single end-to-end pipeline. The pipeline takes a natural language query, finds the most semantically similar documents using DistilBERT embeddings, and enriches each result with extracted named entities.

This integration directly previews the retrieval-augmented systems you will build in Module 8.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Learning Objectives

After completing this integration, you will be able to:

- Build an end-to-end NLP pipeline that combines entity extraction and semantic representations
- Run NER (Named Entity Recognition) on a text corpus and store structured entity results
- Compute DistilBERT embeddings for document-level semantic search
- Implement cosine-similarity-based semantic search that returns ranked results
- Enrich search results with extracted entities for richer, more informative output
- Compare semantic search to keyword-based search and articulate the differences

Setup

1. [Accept the assignment](#)
2. Clone your repo and open it in VS Code.
3. The repo contains:

- `data/climate_articles.csv` – the same climate articles dataset from both weeks
- `data/example_queries.txt` – 3 example search queries to demonstrate your pipeline

4. **Install dependencies:**

```
pip install -r requirements.txt
```

`transformers` requires PyTorch for the DistilBERT embedding tasks. `torch` is listed in `requirements.txt` (version 2.2 or higher), and `pip install` will download the correct version for your computer automatically. Intel Macs need 2.2.x because PyTorch dropped x86_64 macOS wheels at 2.4; Apple Silicon and Linux learners pick up a newer release. This step also installs `transformers`, `spacy`, `scikit-learn`, `pandas`, `numpy`, and `pytest`. If you have not already downloaded the spaCy English model, run:

```
python -m spacy download en_core_web_sm
```

5. **Create a working branch:**

```
git checkout -b integration-6b-semantic-pipeline
```

Tasks

Complete the implementation of `semantic_pipeline.py`. The file contains function signatures with docstrings and a structured script layout. You implement the logic within each function.

Task 1 – Load and Preprocess the Corpus

Implement `load_and_preprocess(filepath)`:

- Load `data/climate_articles.csv` using `pandas`
- Drop rows with missing `text` values; ensure the `text` column is clean
- Recommended: filter to English-language texts (the `language` column, same as Week A) — the rest of the pipeline uses spaCy `en_core_web_sm` and English-only DistilBERT, so Arabic rows produce poor entities and embeddings. The autograder does not enforce the filter, but filtering keeps the pipeline coherent
- Return the resulting `DataFrame` (must include a `text` column)

This is the same dataset you used in Week A. You are building on top of the preprocessing and NER work from that week.

Task 2 – Run NER on All Texts

Implement `run_ner(texts)`:

- Load the spaCy model (`en_core_web_sm`) inside the function
- For each text, run the spaCy NLP pipeline and extract named entities
- Return a `pandas DataFrame` with columns `text_index` (integer position in `texts`), `entity_text` (the entity string), and `entity_label` (e.g., `ORG`, `GPE`, `PERSON`). One row per extracted entity

This reuses the NER skills from Week A. The `DataFrame` is joined back to search results in Task 5.

Note on NER quality: spaCy's `en_core_web_sm` is a small, general-purpose model — its labels are useful but imperfect. You may see entities like "Sub-Saharan Africa" tagged as `ORG` instead of `LOC`, or multi-word organizations split inconsistently. This is expected: production pipelines often add a custom NER step or a larger model (`en_core_web_lg`) for domain-specific text. Treat the entity output as a useful enrichment, not ground truth.

Task 3 – Compute DistilBERT Embeddings

Implement `compute_embeddings(texts, tokenizer, model)`:

- Use the injected `tokenizer` and `model` parameters (DistilBERT). They are loaded once in `__main__` and passed in — do not load them again inside this function
- For each text, extract the sentence embedding using mean pooling (same pipeline as the lab)
- Return a `numpy array` of shape `(n_texts, 768)`

Task 4 – Implement Semantic Search

Implement `semantic_search(query, corpus_embeddings, corpus_texts, top_k=5)`:

- `query` is a **pre-computed query embedding** (`numpy array` of shape `(768,)`), not a raw text string. The orchestrator function (Task 6) is responsible for embedding the query first
- `corpus_embeddings` is the `(n, 768)` array from Task 3; `corpus_texts` is the parallel list of text strings
- Compute cosine similarity between the query embedding and every corpus embedding
- Return a list of `(text, similarity_score)` tuples for the top-`top_k` most similar documents, sorted by similarity descending

Task 5 – Enrich Results with Entities

Implement `enrich_with_entities(search_results, entity_df, corpus_texts)`:

- `search_results` is the list of `(text, score)` tuples returned by `semantic_search`
- `entity_df` is the `DataFrame` returned by `run_ner` (columns: `text_index`, `entity_text`, `entity_label`)
- `corpus_texts` is the original list of strings passed to `run_ner` — use it to map a result text back to its `text_index` (e.g., `corpus_texts.index(result_text)`). This is required because `entity_df` is keyed by integer position, not by the text string itself
- For each search result, look up its `text_index`, filter `entity_df` to that index, and attach the matching entities
- Return a list of dictionaries, one per result, with keys:
 - `text`: the result text
 - `similarity`: the cosine similarity score
 - `entities`: a list of `["text": ..., "label": ...]` dicts, one per extracted entity

This is the data-preparation step. Pretty-printing happens in `__main__` once `demonstrate_pipeline` (Task 6) returns the enriched dictionary.

Task 6 – Orchestrate the Pipeline

Implement `demonstrate_pipeline(corpus_df, entity_df, embeddings, queries, tokenizer, model)`:

- Use the injected `tokenizer` and `model` parameters. `__main__` already loaded DistilBERT once and passes them in — do not load them again inside this function
- For each query string in `queries`:
 1. Compute the query embedding by calling `compute_embeddings([query], tokenizer, model)[0]` (the same mean-pooling pipeline as Task 3)
 2. Call `semantic_search(query_emb, embeddings, corpus_df["text"].tolist())`
 3. Call `enrich_with_entities(search_results, entity_df, corpus_df["text"].tolist())`
- Return a dictionary mapping each query string to its enriched results list

The `__main__` block iterates the returned dictionary and prints each query's enriched results — text preview (first 100 chars), score, and the first few entities. Format the printed output for readability.

Task 7 – Demonstrate and Report

In `__main__`, the script runs the pipeline on the 3 example queries from `data/example_queries.txt` and prints the results. In your PR description, write a structured report:

1. **Pipeline demo output** for all 3 queries: paste the formatted output produced by `__main__` after `demonstrate_pipeline` returns (Task 6)
2. **Semantic vs. keyword comparison:** For at least one query, compare the top-5 results from your semantic search to a simple keyword search (e.g., checking if the query words appear in the text). Where does semantic search find relevant results that keyword search misses?
3. **Entity enrichment value:** How do the displayed entities help a user evaluate search results? Give a specific example from your output.
4. **Production improvements:** Propose 2-3 specific improvements for deploying this pipeline in production (consider speed, scalability, result quality, or user experience)

Running Tests Locally

Run the autograder tests with:

```
pytest tests/ -v
```

Note on BERT tests: This assignment uses PyTorch and Hugging Face transformers for embedding computation and semantic search. `torch` is listed in `requirements.txt`, so all tests will run locally after `pip install -r requirements.txt`.

If you see some tests marked as **skipped** (the embedding and search tests), it means torch is not in your current environment. The NER and entity enrichment tests will still run and give you feedback. To get full local coverage, run:

```
pip install -r requirements.txt
```

On CI (when you push), all tests always run — the autograder workflow installs the same `requirements.txt` before running `pytest`.

Submission

Open a pull request from `integration-6b-semantic-pipeline` to `main`. Your PR must include:

1. `semantic_pipeline.py` – complete, runnable script (`python semantic_pipeline.py` produces demo output)
2. PR description with:
 - Pipeline demo output for 3 queries
 - Semantic vs. keyword search comparison
 - Entity enrichment analysis
 - Production improvement proposals
3. Paste your PR URL into TalentLMS -> Module 6 Week B -> Integration 6B to submit this assignment.

Note: This is the final deliverable for Module 6. It integrates skills from both weeks: text preprocessing, NER extraction (Week A), embedding computation, semantic similarity, and representation comparison (Week B). The pipeline pattern – embed, search, enrich – is the foundation of modern retrieval systems.

Challenge Extensions

These are optional extensions for learners who finish the base requirements early. They are not required for credit but build toward production NLP engineering.

Tier 1 – Hybrid Search

Semantic search captures meaning but can miss exact keyword matches. Keyword search finds exact terms but misses synonyms and paraphrases.

- Implement a TF-IDF keyword search alongside your semantic search
- Build a hybrid scoring function: `hybrid_score = alpha * semantic_score + (1 - alpha) * tfidf_score` where `alpha` is a configurable weight
- Run the 3 example queries with `alpha` values of 0.3, 0.5, 0.7, and 1.0
- For each alpha, show the top-5 results and note which results change position
- Write a brief analysis: for which types of queries is hybrid search better than pure semantic search?

Tier 2 – Entity-Aware Search

The base task displays entities alongside search results but does not use them to improve search. Entities can serve as structured filters.

- Extend the pipeline to support entity-type filters: "Find documents about climate change that mention organizations"
- Implement a function that first runs semantic search, then filters results to include only documents containing entities of the specified type
- Implement a second function that boosts the similarity score for documents containing entities matching a user-specified entity (e.g., boost results mentioning "IPCC" for climate policy queries)
- Demonstrate both features on the example queries
- Write a paragraph: when would entity-aware search outperform pure semantic search? What are the failure modes?

Tier 3 – Pipeline Architecture for Scale

The base pipeline computes embeddings on-the-fly for every search query. This works for 200 documents but not for 200,000.

- Redesign the pipeline to separate indexing (one-time embedding computation) from querying (search against pre-computed embeddings)
- Implement an `Indexer` class that:
 - Computes and stores embeddings and entities for the entire corpus
 - Saves the index to disk (`numpy arrays` for embeddings, `JSON` for entities)
 - Loads a pre-built index without recomputing
- Implement a `Searcher` class that:
 - Loads a pre-built index
 - Accepts a query and returns enriched results without touching the original model (uses only the stored embeddings)
 - Supports adding new documents to the index without recomputing existing embeddings
- Benchmark the query latency of your pre-computed approach versus the on-the-fly approach
- Write a design document: how would you scale this to 1 million documents? What components would need to change? (This previews the vector database concepts in Module 8.)